

Tecnatura Superior en Desarrollo de Software

Anexo 07b — Jerarquía completa del OutputPort

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Dónde estamos parados?

Lectura previa recomendada:

En el **Módulo 04** conociste el concepto básico de OutputPort: una interfaz con métodos `Task` como `HandleSuccessAsync()` y `HandleValidationErrorAsync()` que el UseCase llama para notificar resultados.

Este anexo toma ese concepto y lo evoluciona hacia una **jerarquía de 4 niveles** que elimina la repetición de código, estandariza el contrato entre capas y hace que cada implementación concreta solo escriba lo que le es propio.

En el Módulo 04, el OutputPort de un UseCase se veía así — simple y directo, pero con una limitación importante:

```
1 // patrón simple – Módulo 04
2 public interface IPlaceOrderOutputPort
3 {
4     Task HandleSuccessAsync(PlaceOrderResponse response);
5     Task HandleValidationErrorAsync(string message);
6 }
7
8 // la limitación: CADA Presenter que implemente
9 // esta interfaz debe escribir TODOS los métodos desde cero
10 // no hay reutilización – no hay contrato compartido entre UseCases
```

A medida que el sistema crece, esta limitación se vuelve un problema real: el proyecto tendrá decenas de UseCases, cada uno con su OutputPort. Si cada implementación concreta repite los mismos métodos base, el código se duplica y

cualquier cambio en el contrato base requiere modificar todos los OutputPorts del sistema.

La jerarquía de 4 niveles

Evolución del OutputPort simple

Capa: Application + InterfaceAdapter

La solución es organizar el OutputPort en cuatro niveles con responsabilidades bien definidas. Cada nivel agrega algo específico sin repetir lo que el nivel anterior ya definió.

El resultado es que las implementaciones concretas — distintos Presenters, dobles de prueba (fakes) — solo escriben el código que les es propio. El comportamiento compartido vive en la clase base abstracta y se hereda automáticamente.

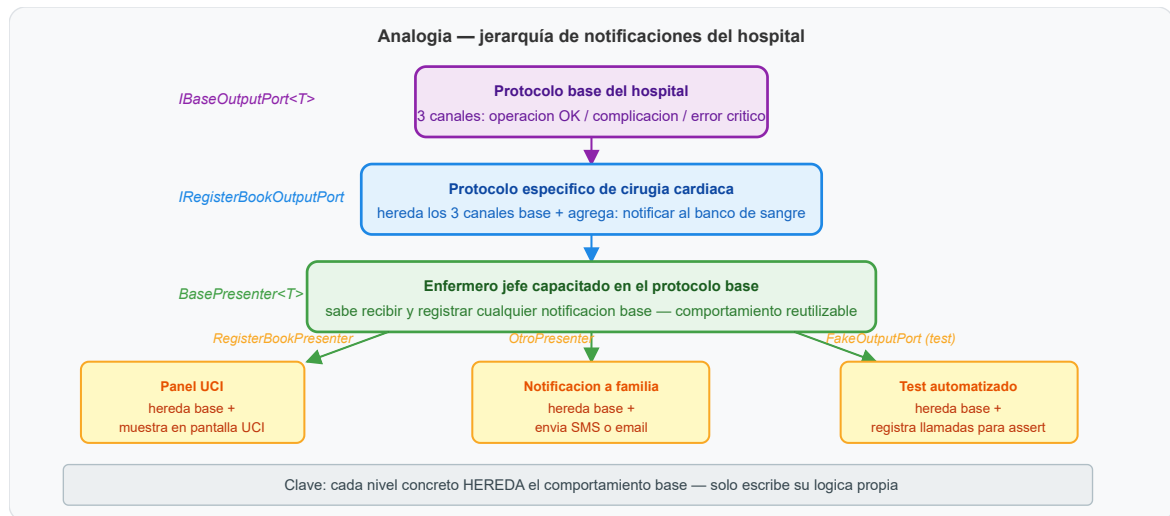
Los 4 niveles:

- Nivel 1 — IBaseOutputPort<T>
- Nivel 2 — IRegisterBookOutputPort
- Nivel 3 — BasePresenter<T>
- Nivel 4 — RegisterBookPresenter

Analogía del mundo real

Imaginá el sistema de notificaciones de un hospital de alta complejidad. Cuando una cirugía termina, el quirófano debe notificar a múltiples destinatarios: familiares, enfermería, farmacia, UCI. Cada uno necesita recibir la misma notificación base — *salió bien, hay una complicación, hubo un error* — pero también notificaciones específicas del tipo de cirugía.

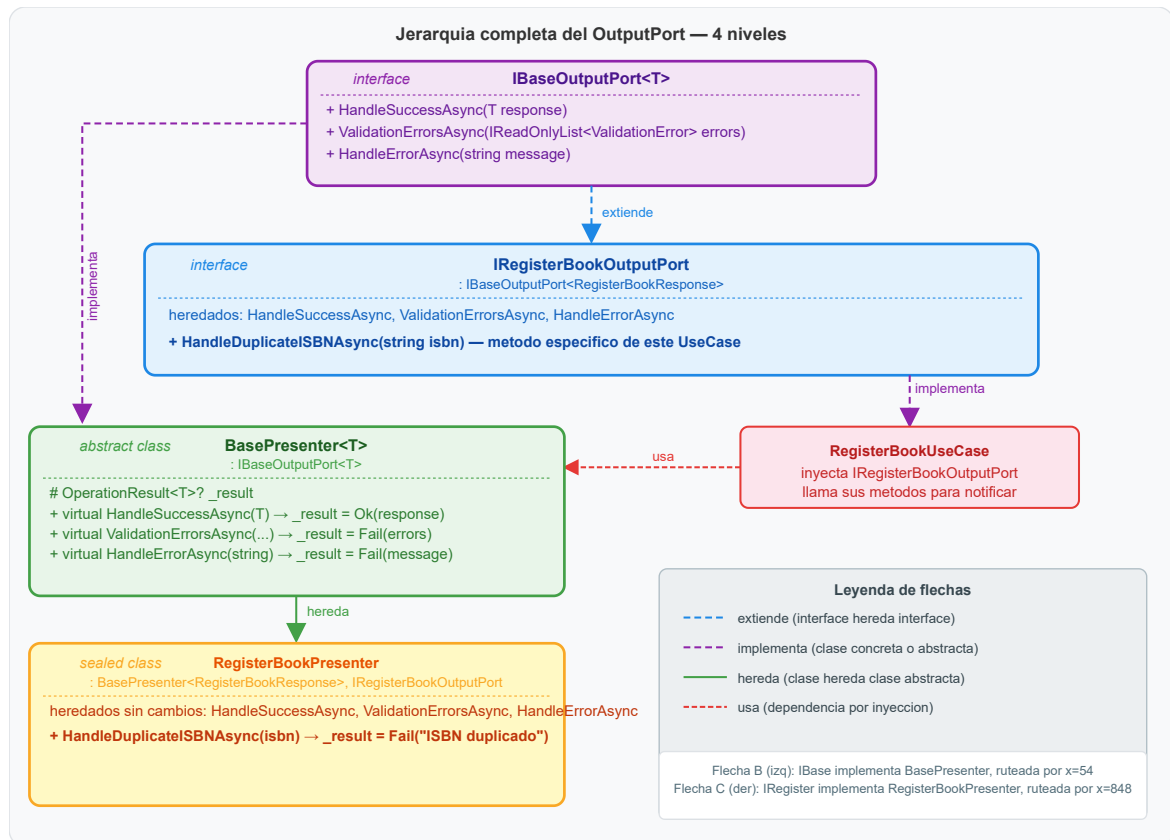
Para no reinventar el sistema de comunicación en cada quirófano, el hospital define una jerarquía:



Trasladando la analogía al código: el "protocolo base del hospital" es `IBaseOutputPort<T>`, el "protocolo de cirugía cardíaca" es `IRegisterBookOutputPort`, el "enfermero jefe capacitado" es `BasePresenter<T>` y cada destinatario concreto (*Panel UCI*, *familia*, *test*) es una implementación que hereda de `BasePresenter<T>` e implementa la interfaz específica.

Los cuatro niveles — visión de conjunto

Antes de ver cada nivel en detalle, observá el diagrama completo de la jerarquía con sus relaciones de herencia, implementación y uso:



Resumen de ubicaciones:

IBaseOutputPort<T> → Application/Common/Ports/IBaseOutputPort.cs

IRegisterBookOutputPort → Application/Books/Ports/IRegisterBookOutputPort.cs

BasePresenter<T> → InterfaceAdapter/Common/Presenters/BasePresenter.cs

RegisterBookPresenter → InterfaceAdapter/Books/RegisterBookPresenter.cs

Errores comunes

Error 1 — inyectar el OutputPort concreto en lugar de la interfaz

ERROR COMUN

UseCase recibe la clase concreta — acoplado a la implementación concreta, inutilizable en tests.

```
1 public RegisterBookUseCase(RegisterBookPresenter vm)
2     => _outputPort = vm; // acoplado a Blazor
```

BUENA PRACTICA

UseCase recibe la interfaz — el contenedor DI decide qué implementación usar.

```
1 public RegisterBookUseCase(IRegisterBookOutputPort outputPort)
2     => _outputPort = outputPort; // independiente del receptor
```

Error 2 — no llamar a base.HandleSuccessAsync() al sobrescribir

ERROR COMUN

Override sin llamar a base — _result nunca se actualiza, el ViewModel no puede leer el resultado.

```
1 public override Task HandleSuccessAsync(RegisterBookResponse r)
2 {
3     // MAL: no llama a base — _result permanece null
4     RegisteredBookId = r.BookId;
5     return Task.CompletedTask;
6 }
```

BUENA PRACTICA

Siempre llamar a base primero — garantiza que `_result` quede actualizado.

```
1 public override Task HandleSuccessAsync(RegisterBookResponse r)
2 {
3     base.HandleSuccessAsync(r); // _result = Ok(r)
4     RegisteredBookId = r.BookId;
5     return Task.CompletedTask;
6 }
```

Error 3 — agregar métodos de negocio a IBaseOutputPort**ERROR COMUN**

Métodos específicos en la base — obliga a TODOS los OutputPorts del sistema a implementarlo aunque no tengan ISBN.

```
1 // MAL: IBaseOutputPort cargado con métodos específicos
2 public interface IBaseOutputPort<T>
3 {
4     Task HandleSuccessAsync(T response);
5     Task ValidationErrorsAsync(IReadOnlyList<ValidationError> errors);
6     Task HandleErrorAsync(string message);
7     // MAL: HandleDuplicateISBNAsync no pertenece al contrato base
8     Task HandleDuplicateISBNAsync(string isbn);
9     // MAL: todos los OutputPorts del sistema ahora deben implementarlo
10    // incluso OrderOutputPort, CustomerOutputPort, InvoiceOutputPort, etc.
11 }
```

BUENA PRACTICA

Los métodos específicos van en la interfaz propia del UseCase — IBaseOutputPort queda limpio y universal.

```
1 // IBaseOutputPort tiene solo los 3 canales universales
2 public interface IBaseOutputPort<T>
3 {
4     Task HandleSuccessAsync(T response);
5     Task ValidationErrorsAsync(IReadOnlyList<ValidationError> errors);
6     Task HandleErrorAsync(string message);
7 }
8
9 // IRegisterBookOutputPort extiende la base y agrega solo lo suyo
10 public interface IRegisterBookOutputPort
11     : IBaseOutputPort<RegisterBookResponse>
12 {
13     Task HandleDuplicateISBNAsync(string isbn);
14 }
```

Error 4 — poner lógica de presentación en el UseCase**ERROR COMUN**

El UseCase decide cómo mostrar los datos — viola la separación de responsabilidades y acopla Application a la UI.

```
1 // MAL: el UseCase formatea el mensaje para la UI
2 var msg = $"<b>Libro registrado</b>: ID {book.Id}";
3 await _outputPort.HandleSuccessAsync(msg); // HTML en Application
```

BUENA PRACTICA

El UseCase devuelve datos crudos; la presentación se decide después, fuera de Application.

```
1 // UseCase: entrega datos sin formato
2 var response = new RegisterBookResponse { BookId = book.Id };
3 await _outputPort.HandleSuccessAsync(response);
4
5 // el Presenter solo almacena – el ViewModel (Módulo 07b) es quien formatea
6 // para mostrarlo en la vista, leyendo Presenter.Result
```

Error 5 — el ViewModel da por hecho el éxito sin consultar Presenter.Result**ERROR COMUN**

El ViewModel invoca el UseCase y arma un mensaje fijo — sin saber si realmente hubo éxito.

```
1 // MAL: asume éxito sin mirar el resultado real
2 await _useCase.ExecuteAsync(request);
3 Mensaje = "Libro registrado correctamente"; // ¿y si falló?
```

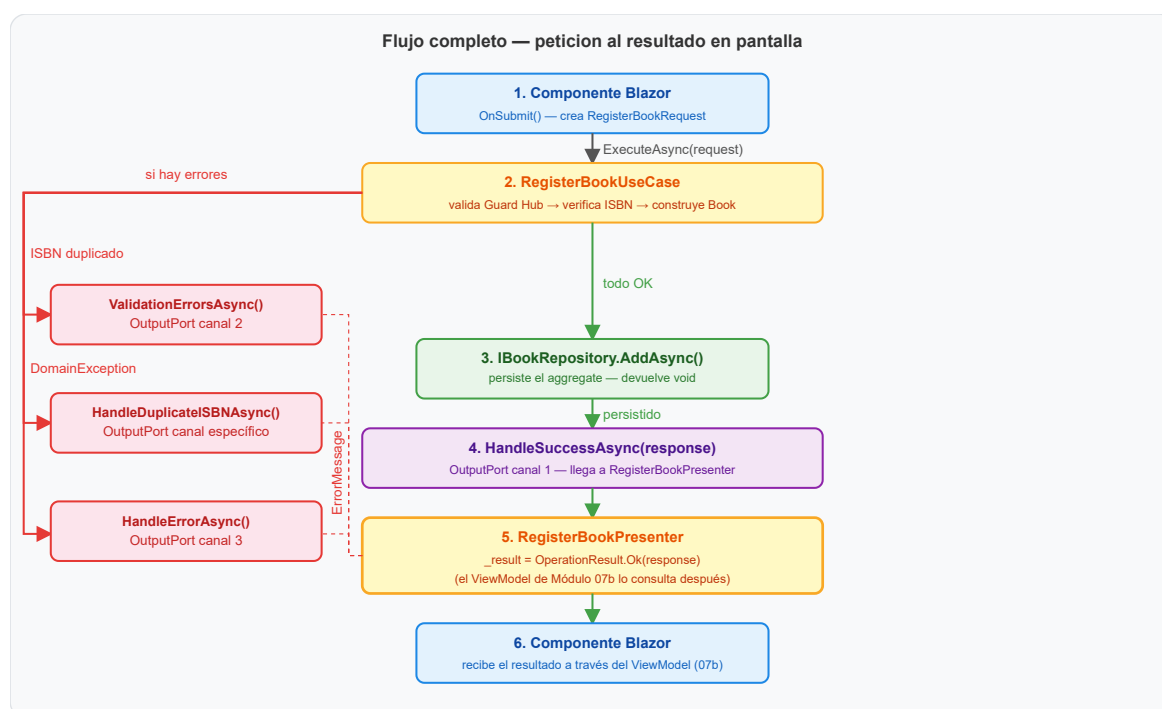
BUENA PRACTICA

El ViewModel consulta `_presenter.Result.IsSuccess` antes de decidir qué exponerle a la vista.

```
1 await _useCase.ExecuteAsync(request);
2
3 // se consulta el resultado antes de decidir qué mostrar
4 var result = _presenter.Result;
5 Mensaje = result.IsSuccess
6     ? "Libro registrado correctamente"
7     : string.Join(", ", result.Errors);
```

Flujo completo — de la petición al resultado en pantalla

El diagrama siguiente resume el recorrido completo de una petición desde que el usuario presiona el botón en el componente Blazor hasta que el resultado aparece en pantalla. Cada flecha muestra quién llama a quién y qué dato transporta.



La flecha de retorno: observá que ninguna flecha vuelve desde el `OutputPort` hacia el `UseCase`. El `UseCase` *notifica y termina*. El componente Blazor no llama directamente al `UseCase`: es el `ViewModel` (Módulo 07b) quien lo hace, y quien ya tiene acceso al `Presenter` por referencia. Después de `ExecuteAsync()`, el `ViewModel` consulta `Presenter.Result` para decidir qué exponerle al componente — sin que el `UseCase` sepa nada de eso. Esto es exactamente la inversión de dependencias en acción.

Resumen de la jerarquía

Nivel	Tipo	Capa	Responsabilidad
1. IBaseOutputPort<T>	interface	Application/Common	Contrato universal — 3 canales para todos los UseCases
2. IRegisterBookOutputPort	interface	Application/Books	Extiende la base — agrega canales específicos del UseCase
3. BasePresenter<T>	abstract class	InterfaceAdapter/Common	Implementa los 3 canales base — almacena <code>OperationResult<T></code>
4. RegisterBookPresenter	sealed class	InterfaceAdapter/Books	Hereda base — implementa canal específico — almacena el resultado en <code>OperationResult<T></code>

La secuencia completa del módulo 06:

06a/06b — Guard: valida cada campo del DTO de forma independiente.

06c — ModelValidatorHub: orquesta todos los Guards — ejecuta todos y acumula errores antes de pasar al UseCase.

06d — OperationResult<T>: encapsula el resultado de la operación (éxito con valor o fallo con mensajes) como un objeto transportable.

06e — OutputPort (este documento): define los canales de comunicación entre el UseCase y el receptor (el Presenter concreto, sea de producción o un doble de test) usando una jerarquía reutilizable que evita duplicación de código.

Estas cuatro piezas trabajan juntas: el Hub produce los errores de validación, el UseCase los entrega al OutputPort, el BasePresenter los guarda en un OperationResult y el Presenter concreto los almacena. El ViewModel (Módulo 07b) es quien finalmente los presenta al usuario.